

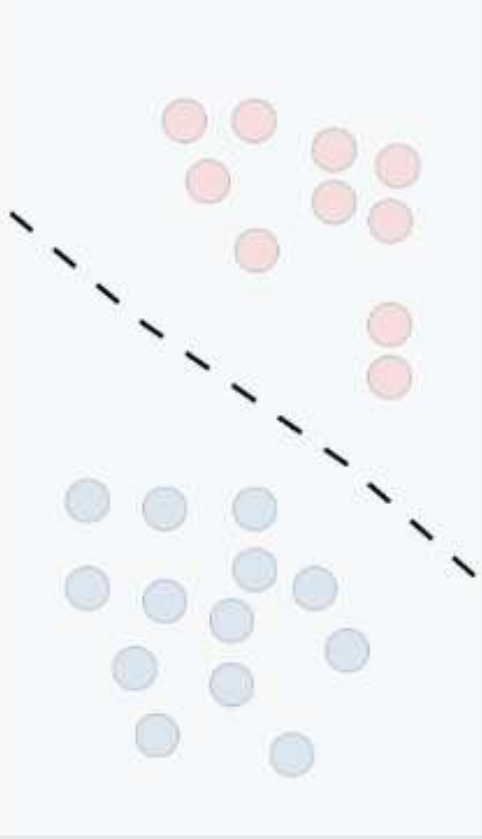
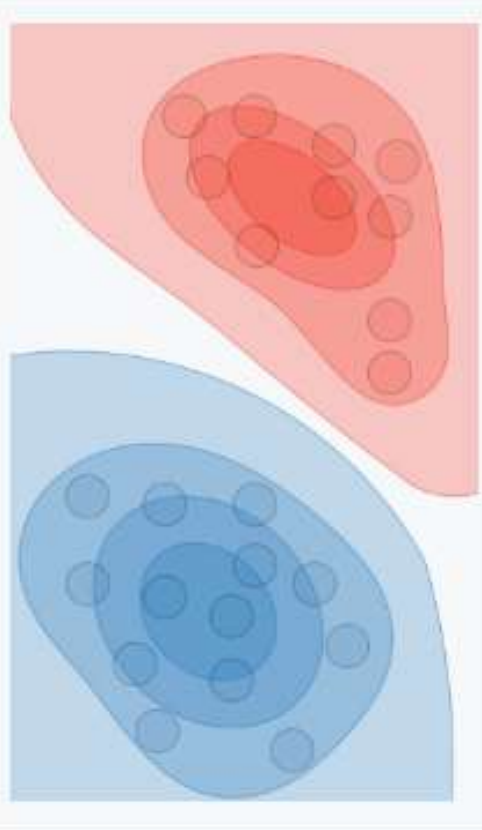
Deep Learning Diffusion

Hao Chen

**Fall 2024
Attendance: @**

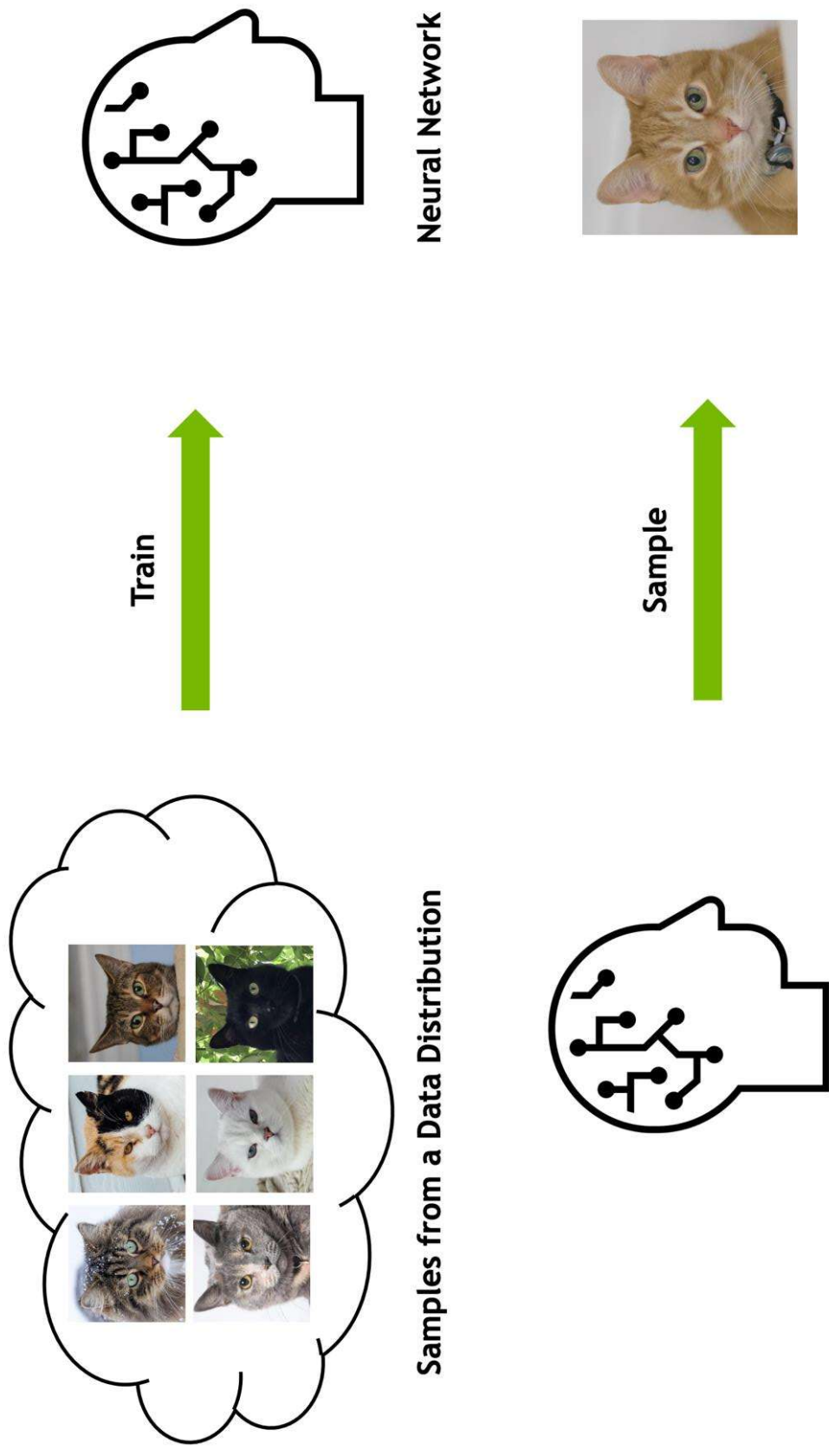
Generative vs. Discriminative

- Generative models learn the data distribution

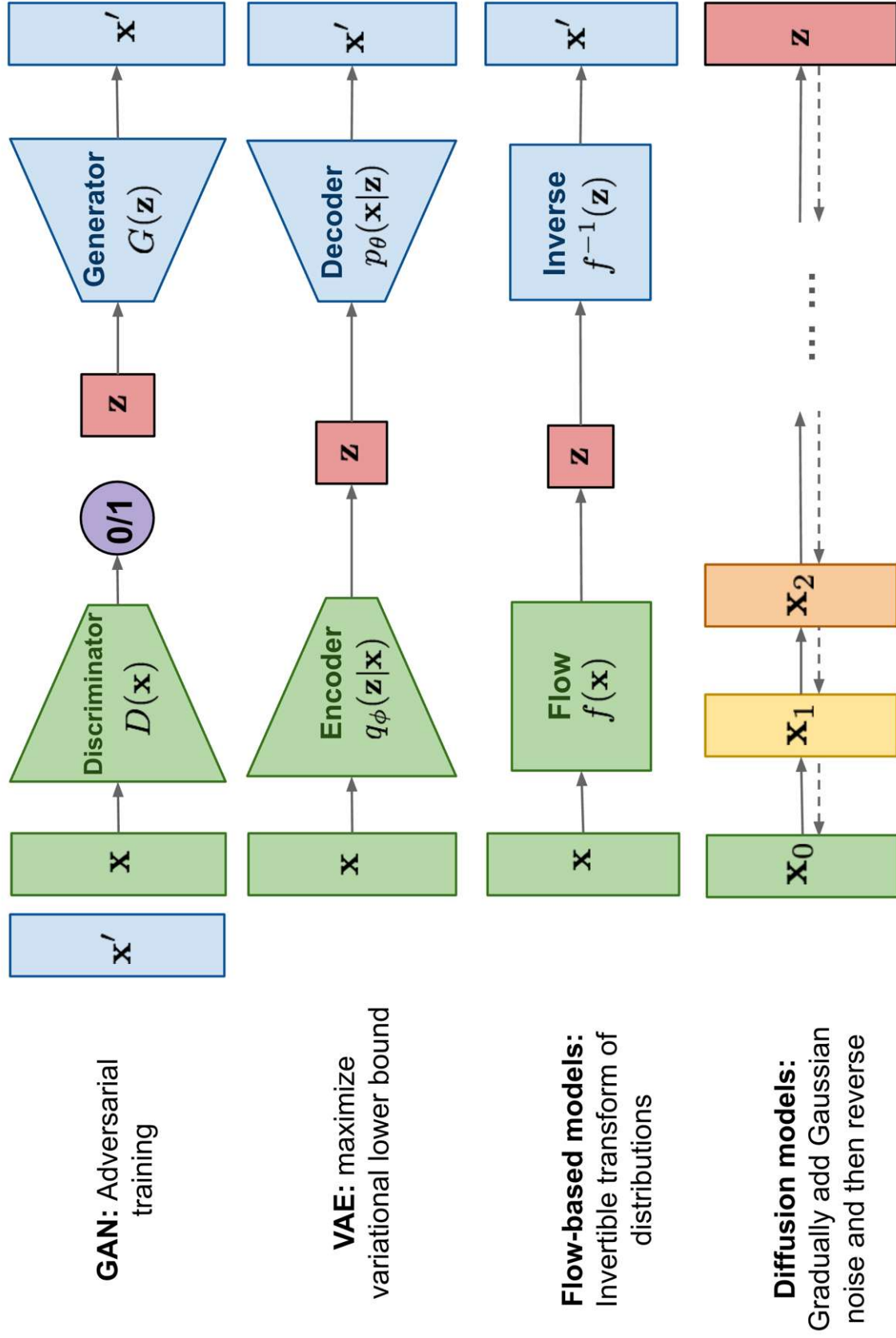
	Discriminative model	Generative model
Goal	Directly estimate $P(y x)$	Estimate $P(x y)$ to then deduce $P(y x)$
What's learned	Decision boundary	Probability distributions of the data
Illustration		

Generative Models

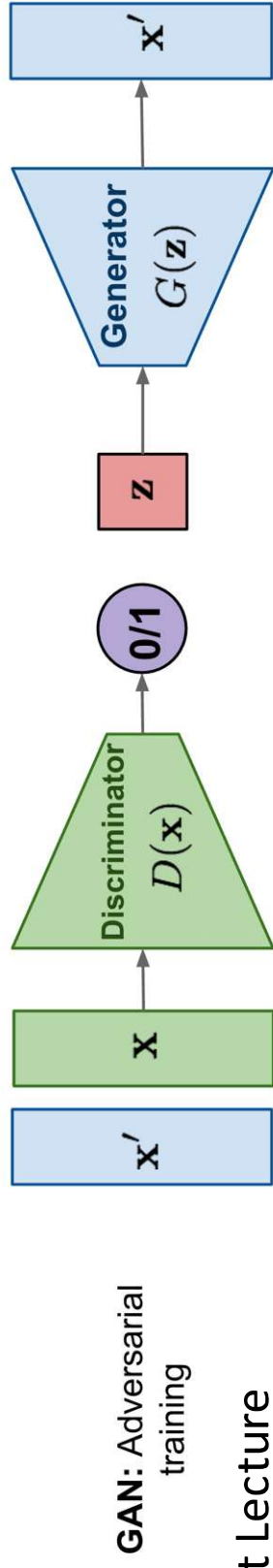
- Learning to generate data



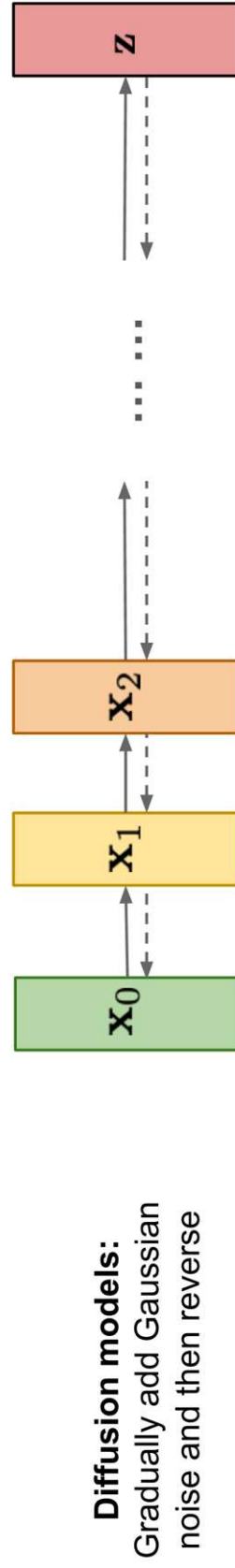
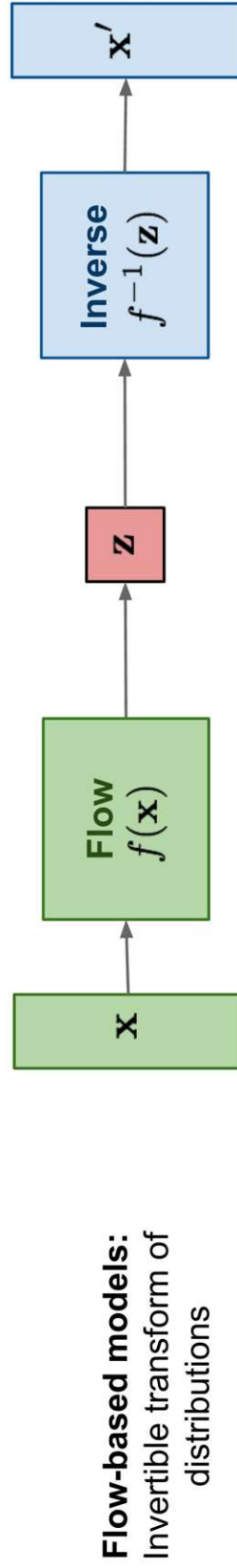
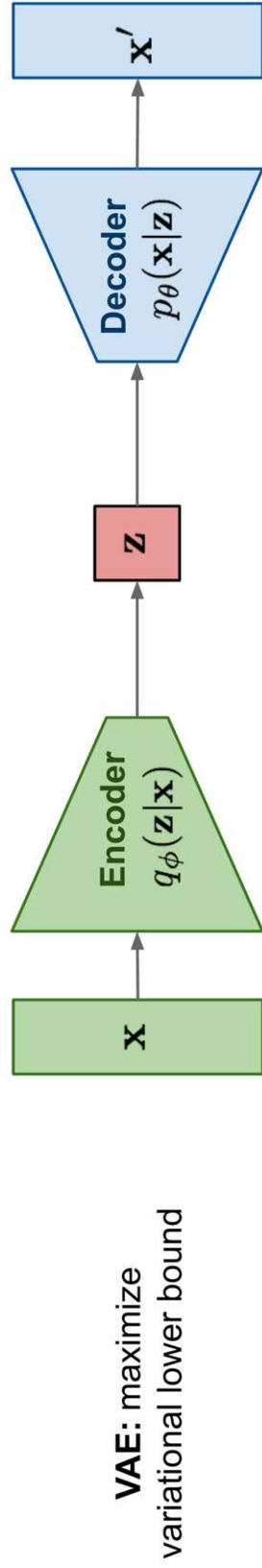
Generative Models



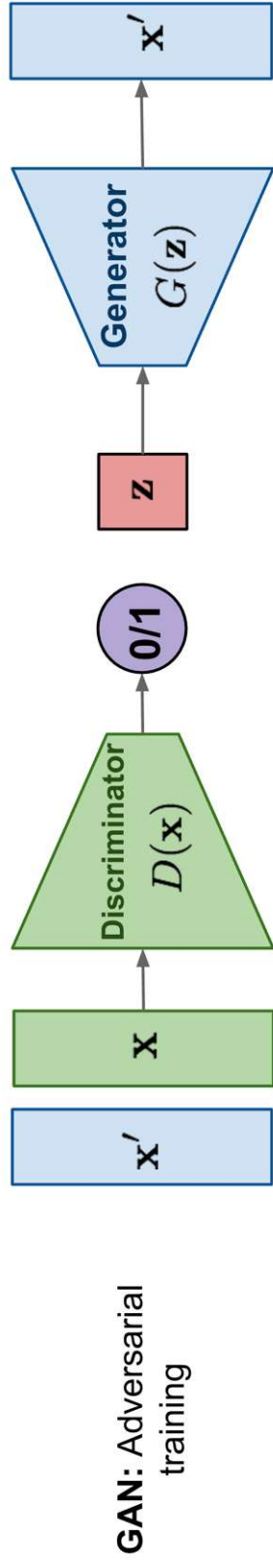
Generative Models



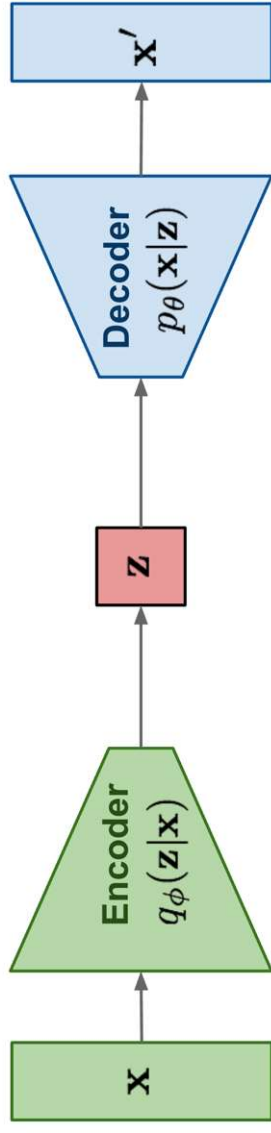
Last Lecture



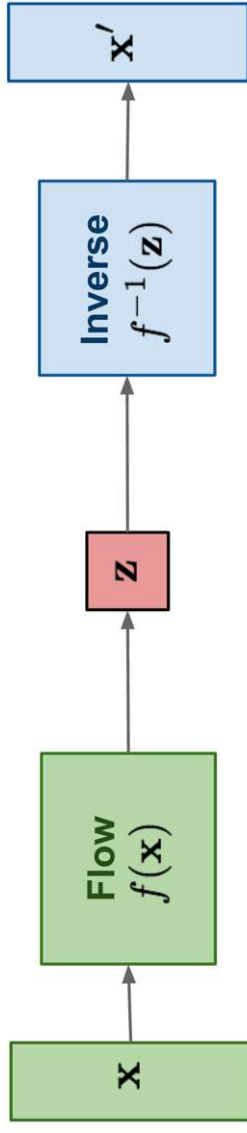
Generative Models



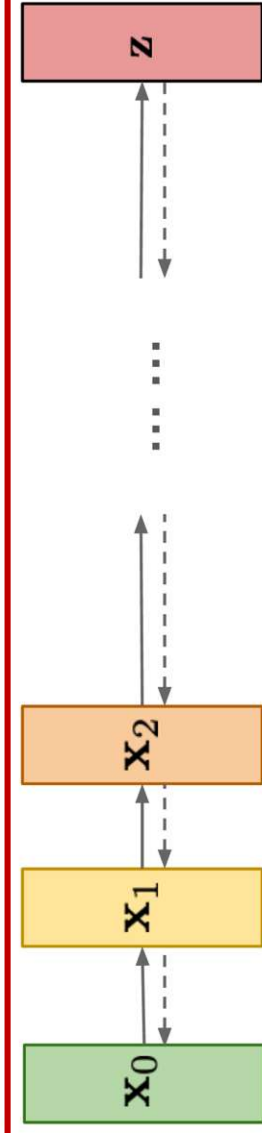
VAE: maximize
variational lower bound



Flow-based models:
Invertible transform of
distributions
This Lecture



Diffusion models:
Gradually add Gaussian
noise and then reverse



A Fast Evolving Field

VAEs, 2013



GANs, 2014



PixelCNN, 2016



BigGAN, 2019



Imagen, 2022



SORA 2024



Content

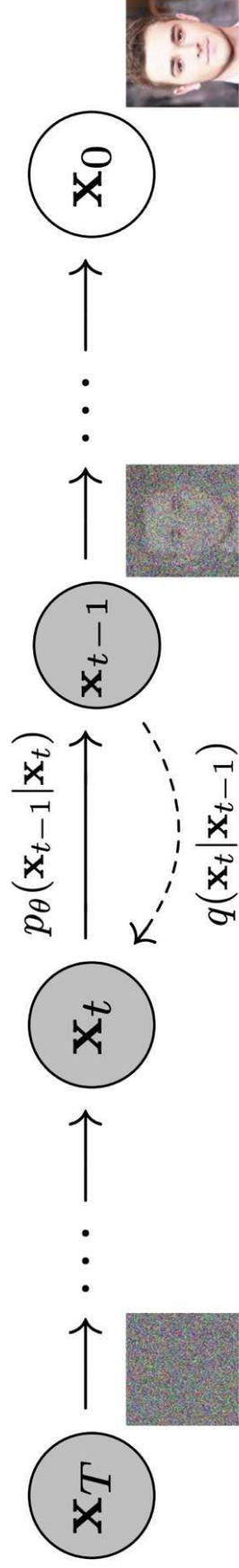
- Denoising Diffusion Model Basics
- Diffusion Models from Stochastic Differential Equations and Score Matching Perspective
- Denoising Diffusion Implicit Model (DDIM)
- Conditional Diffusion Models
- Applications of Diffusion Models

Content

- Diffusion Model Basics
 - Diffusion Models as Stacking VAEs
 - Diffusion Models: Forward, Reverse, Training, Sampling
- Diffusion Models from Stochastic Differential Equations and Score Matching Perspective
- Denoising Diffusion Implicit Model (DDIM)
- Conditional Diffusion Models
- Applications of Diffusion Models

Denoising Diffusion Models

- what we often see about diffusion models



$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

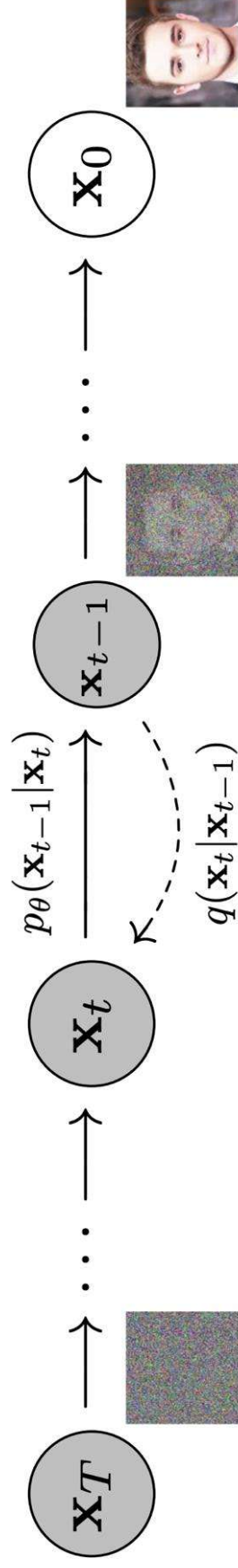
$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

Forward diffusion process

Reverse denoising process

Denoising Diffusion Models

- what we often see about diffusion models



$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

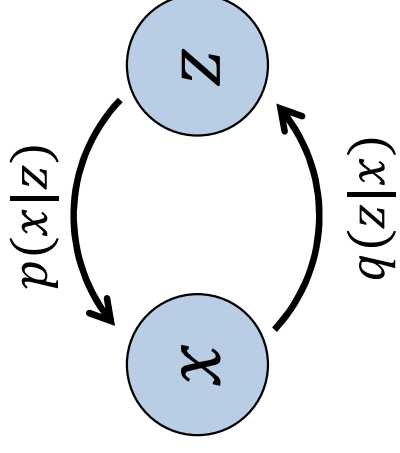
Forward diffusion process

Reverse denoising process

- this lecture: denoising diffusion is a stack of VAEs

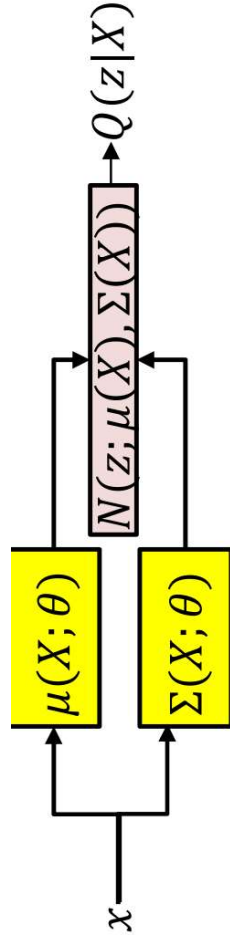
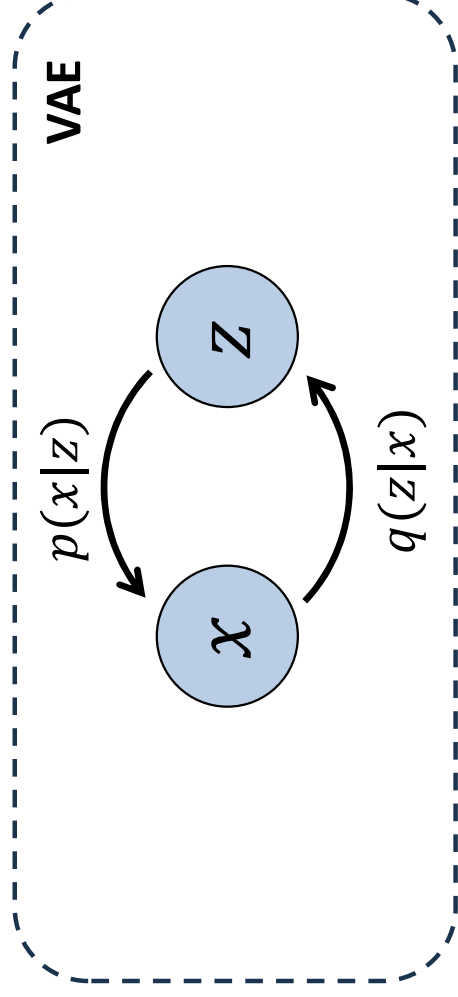
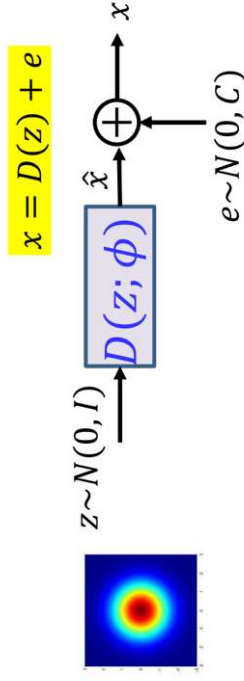
Recap: Variational Autoencoders

- VAEs: a likelihood-based generative model
- **Encoder**: an inference model that approximates the posterior $q(z|x)$
- **Decoder**: a generative model that transforms a Gaussian variable z to real data
- **Training**: maximize the ELBO



Recap: Variational Autoencoders

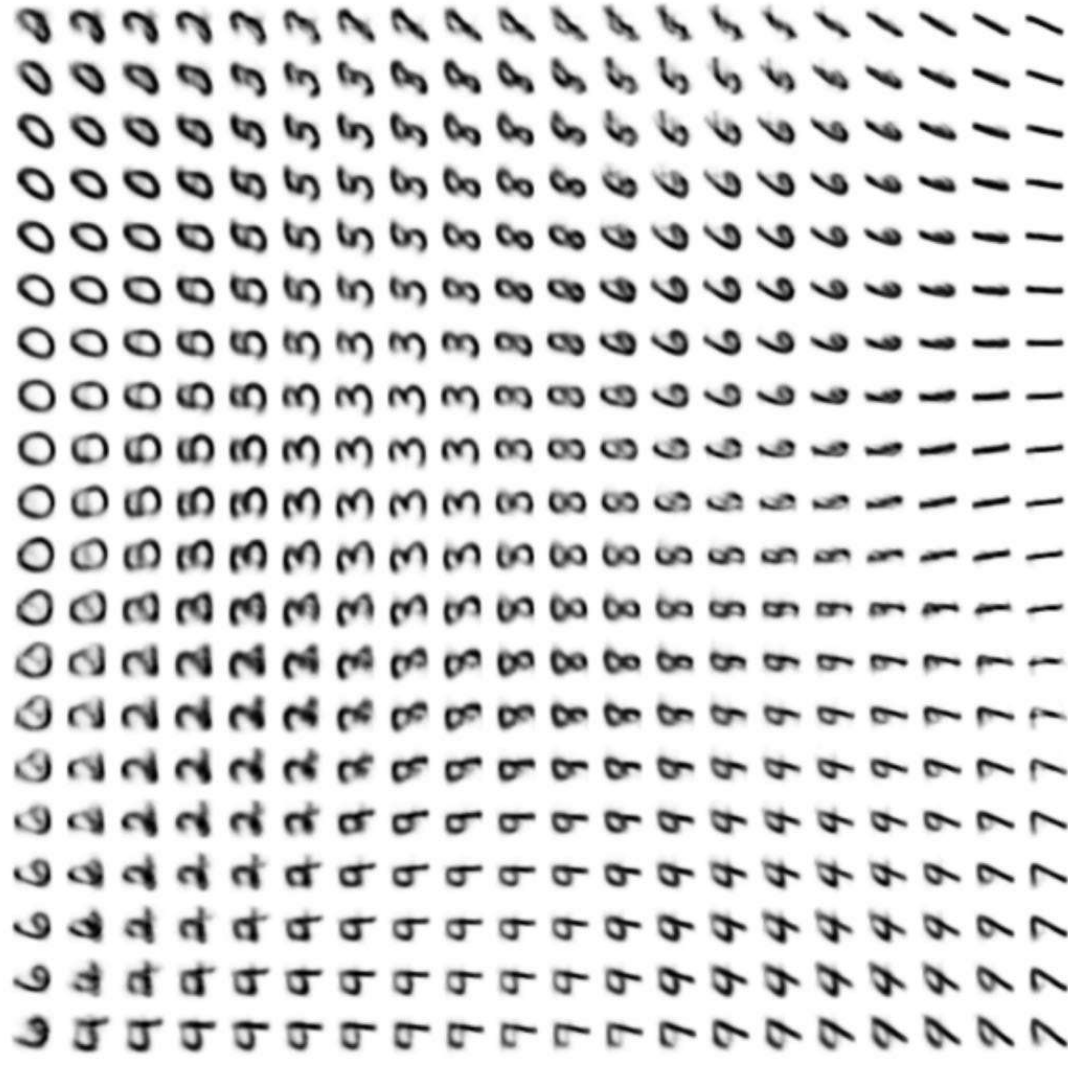
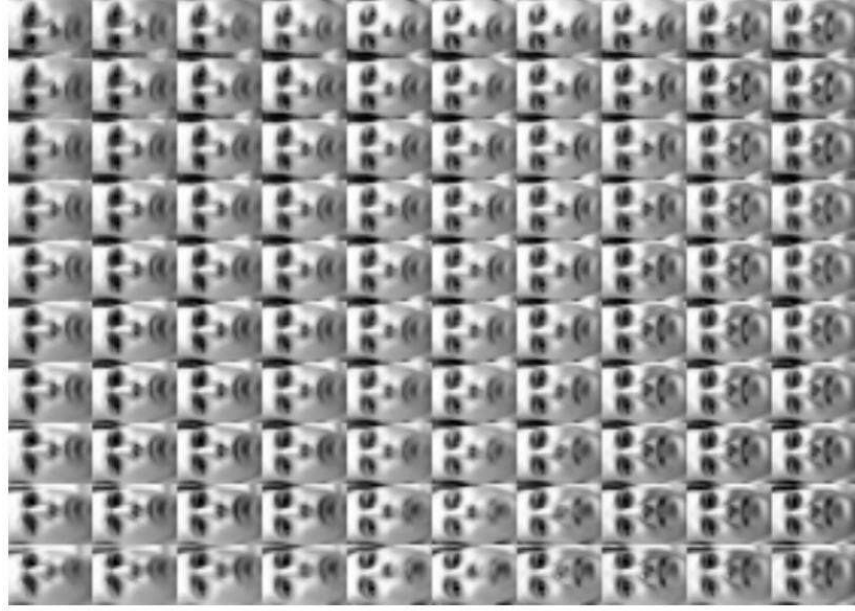
Decoder: transforms a Gaussian variable to real data



Encoder: an inference model approximates the posterior, i.e. Gaussian

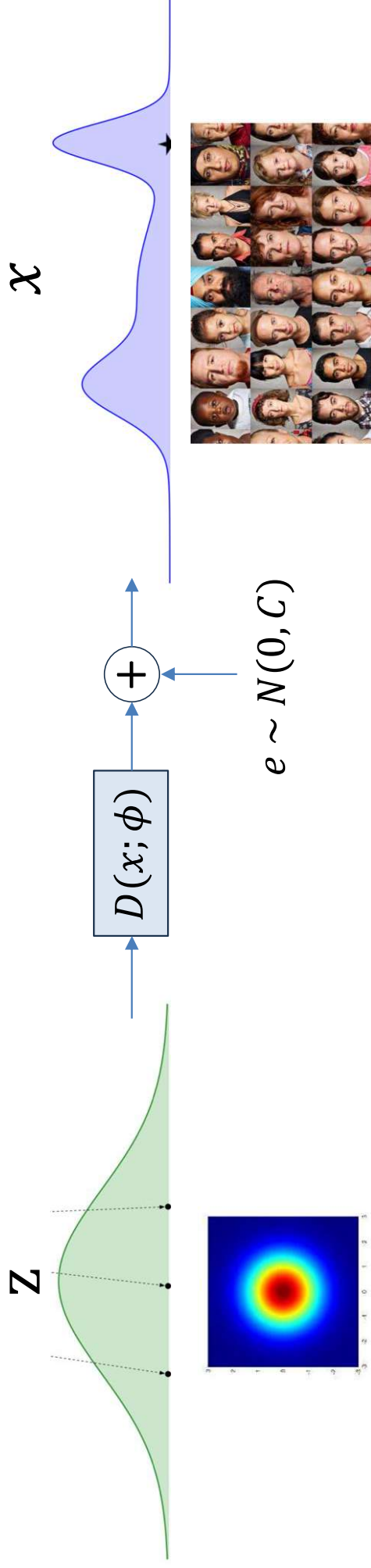
VAEs are good, but...

- Blurry results



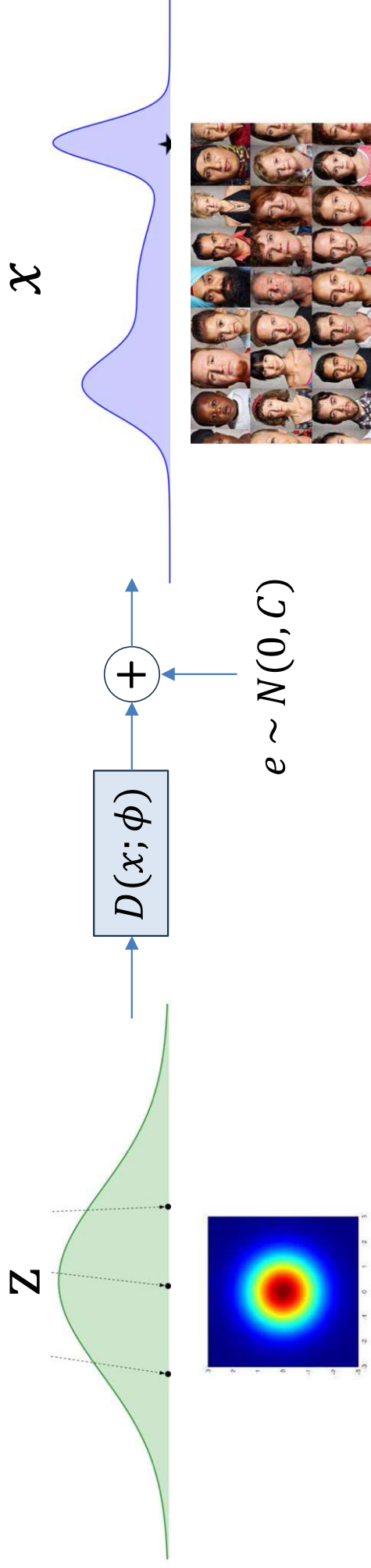
Limitations of VAEs

- Decoder must transform a standard Gaussian all the way to the target distribution in **one-step**
 - Often too large a gap
 - Blurry results are generated



Limitations of VAEs

- Decoder must transform a standard Gaussian all the way to the target distribution in **one-step**
 - Often too large a gap
 - Blurry results are generated



- Solution: have some intermediate latent variables to reduce the gap of each step

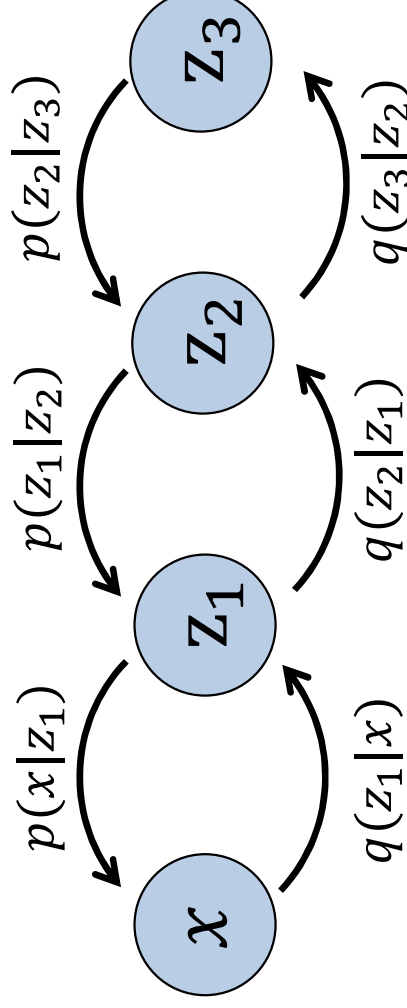
Hierarchical VAEs

- Hierarchical VAEs – Stacking VAEs on top of each other
 - Multiple (T) intermediate latent

- Joint distribution $p(\mathbf{x}, \mathbf{z}_{1:T}) = p(\mathbf{z}_T) p_{\theta}(\mathbf{x} | \mathbf{z}_1) \prod_{t=2}^T p_{\theta}(\mathbf{z}_{t-1} | \mathbf{z}_t)$

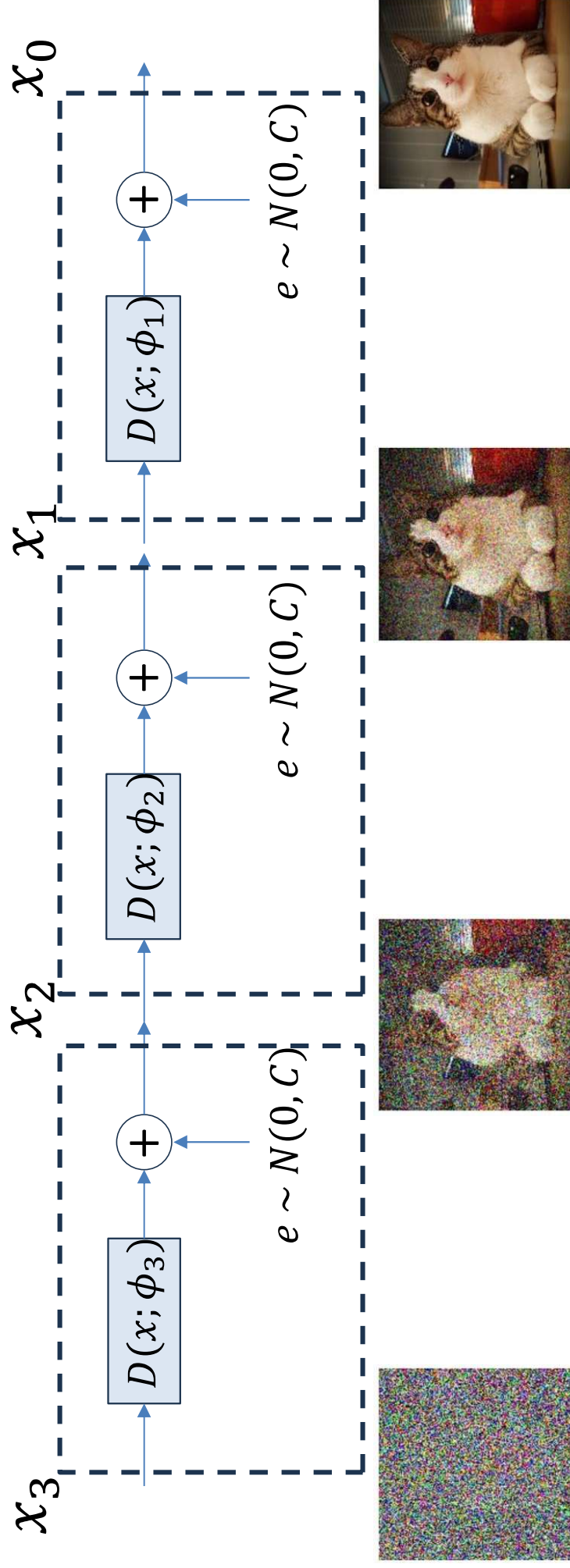
- Posterior $q_{\phi}(\mathbf{z}_{1:T} | \mathbf{x}) = q_{\phi}(\mathbf{z}_1 | \mathbf{x}) \prod_{t=2}^T q_{\phi}(\mathbf{z}_t | \mathbf{z}_{t-1})$

- Better likelihood achieved!



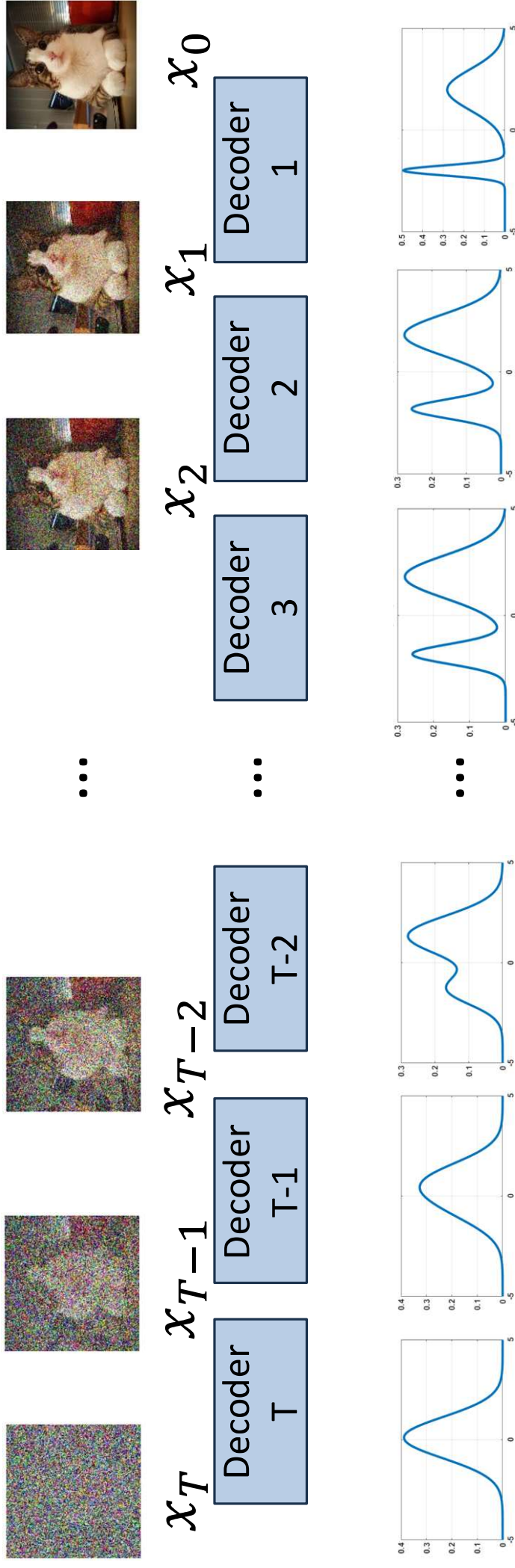
Stacking VAEs

- Each step, the decoder removes part of the noise
- Provides a seed model closer to final distribution



Stacking VAEs

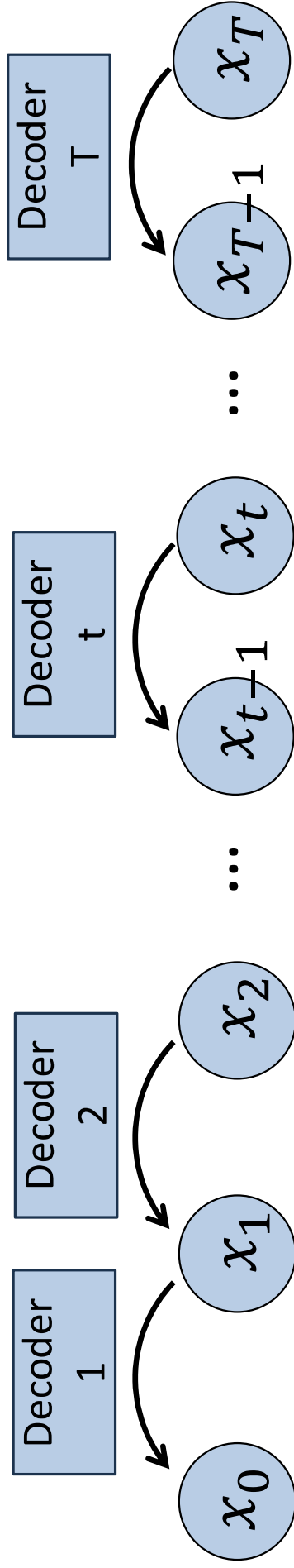
- We can have many many steps (in total T)...
- Each step incrementally recovers the final distribution



- Looks familiar?

Diffusion Models are Stacking VAEs

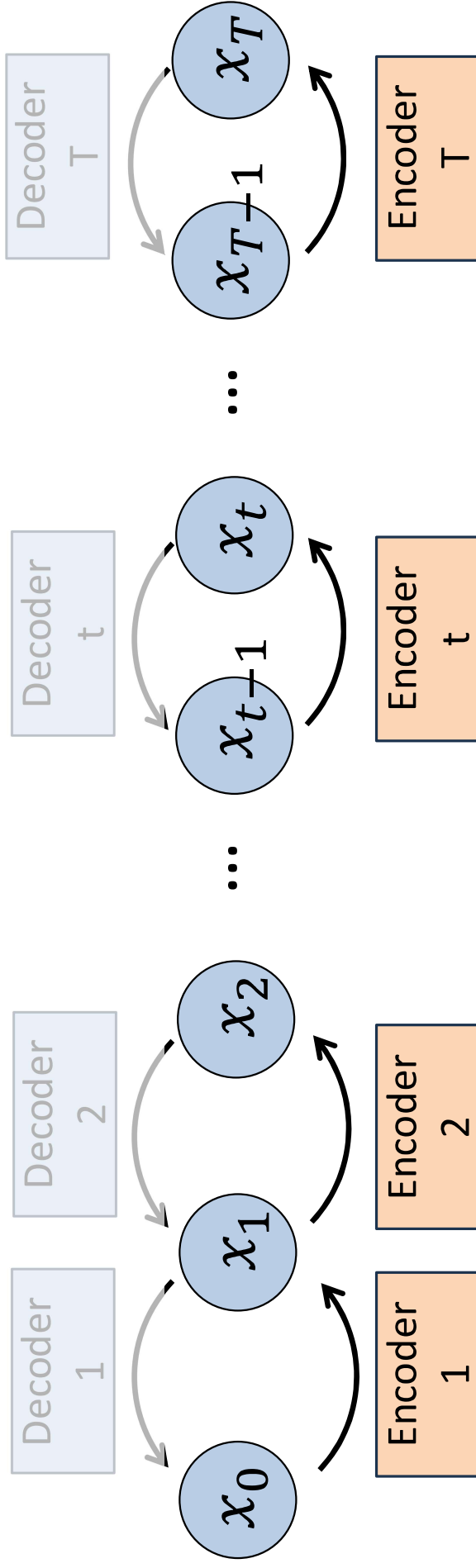
- Diffusion models are special cases of Stacking VAEs



- The reverse denoising process is the stack of decoders
- What about encoders?

Diffusion Models are Stacking VAEs

- Diffusion models are special case of Stacking VAEs



- In VAEs, encoders are learned with KL-divergence between the posterior and the prior
- Suffers from the ‘posterior-collapse’ issue
- Diffusion models use **fixed inference encoders**

Poll

Denoising Diffusion Models

- Diffusion models have two processes
- **Forward diffusion process** gradually adds noise to input
- **Reverse denoising process** learns to generate data by denoising

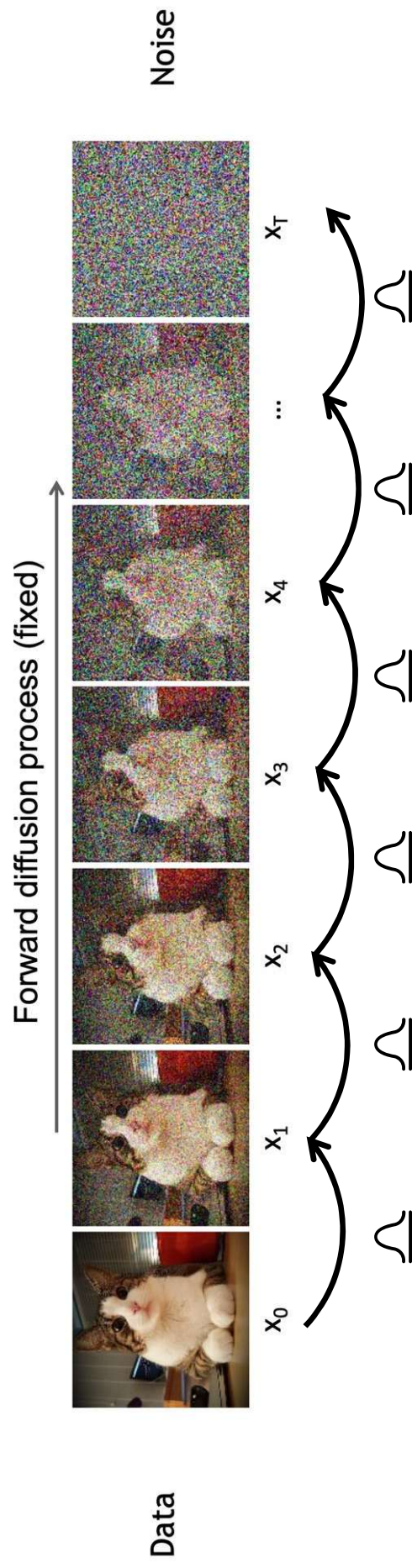


Forward Diffusion Process

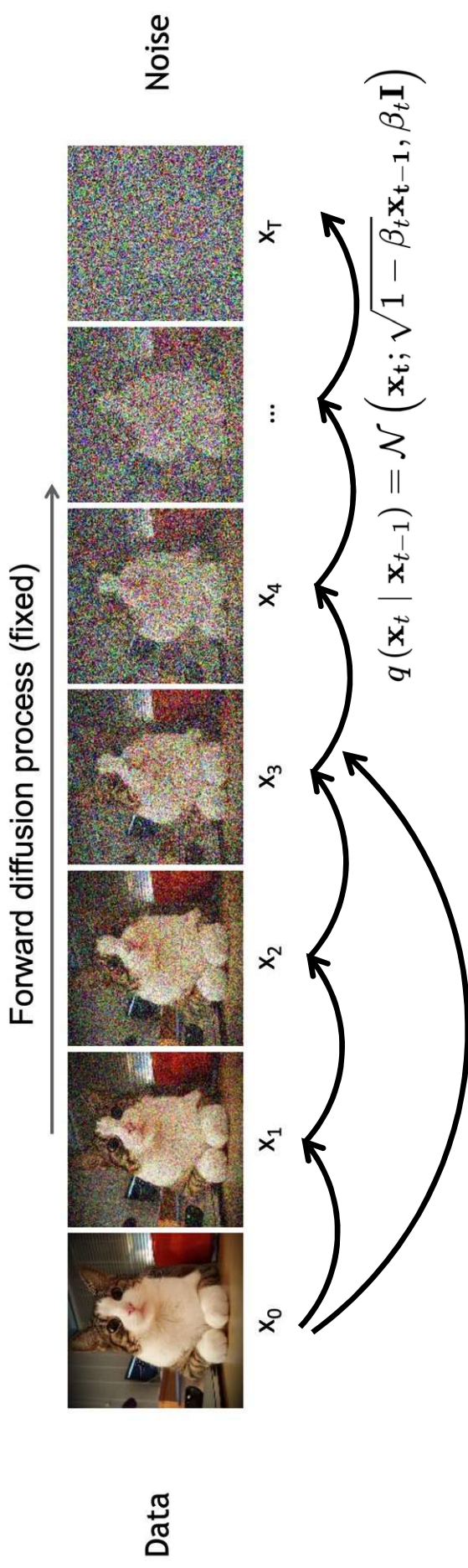
- Forward diffusion process is stacking **fixed** VAE encoders
 - gradually adding Gaussian noise according to schedule β_t

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$



Forward Diffusion Process



- The forward process allows sampling of $\mathbf{x}_t$ at arbitrary timestep t in closed form:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}\right) \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- The noise schedule (β_t values) is designed such that

$$q(\mathbf{x}_T | \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

Reverse Denoising Process

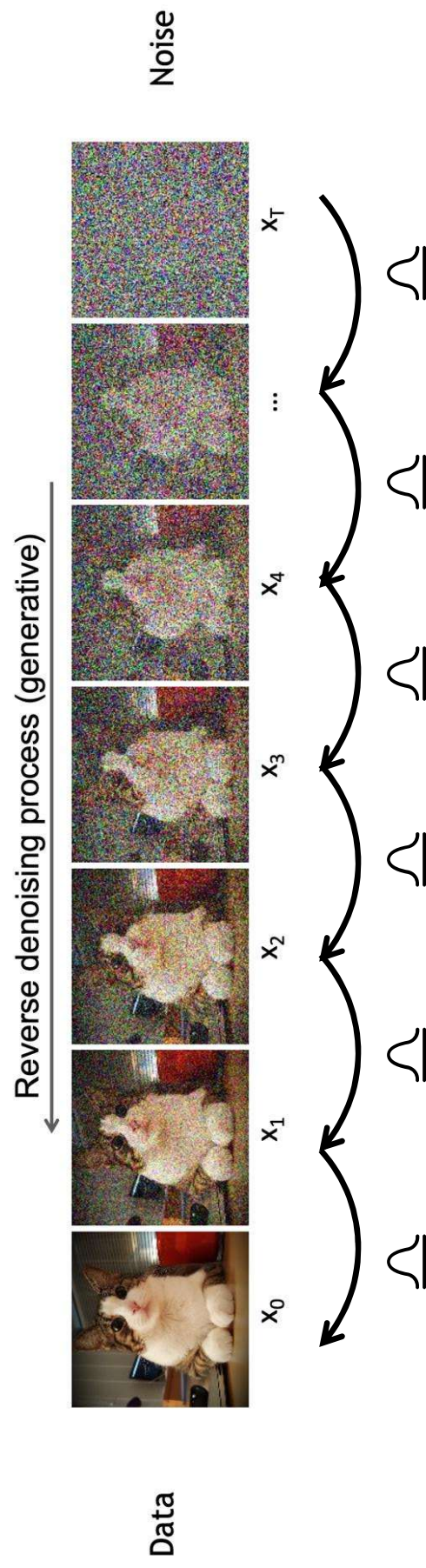
- Generation process
 - Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$
 - Iteratively sample $\mathbf{x}_{t-1} \sim q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$
- $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ not directly tractable
- But can be estimated with a Gaussian distribution if β_t is small at each step
 - The purpose of our stack of VAE decoders!

Reverse Denoising Process

- Reverse diffusion process is stacking **learnable** VAE decoders
 - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \quad p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$$

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

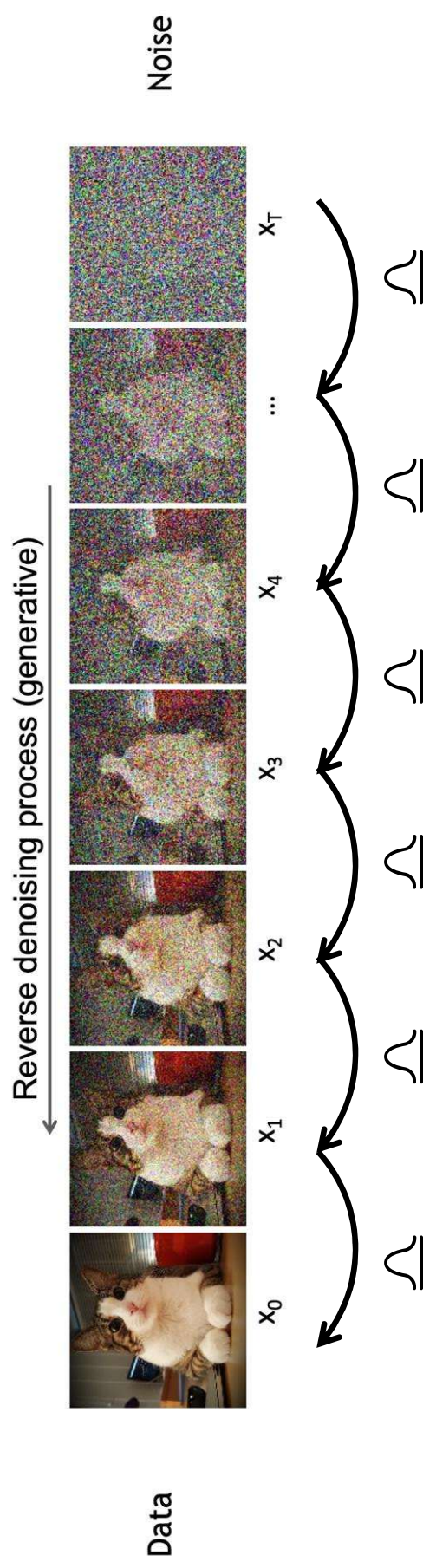


Reverse Denoising Process

- Reverse diffusion process is stacking **learnable** VAE decoders
 - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \quad p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$$

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$



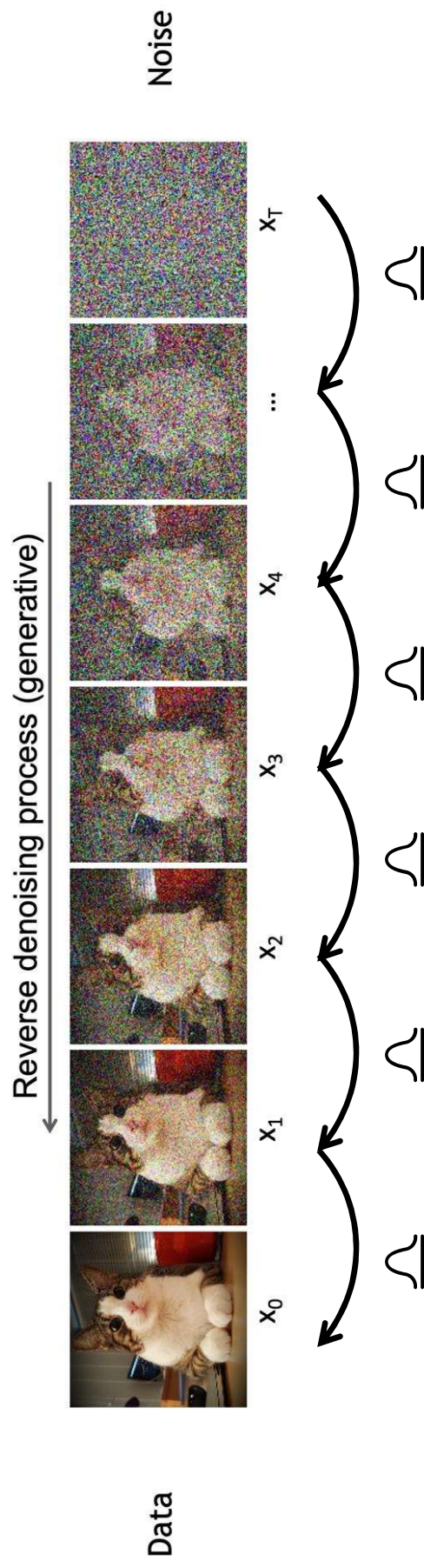
Reverse Denoising Process

- Reverse diffusion process is stacking **learnable** VAE decoders
 - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \quad p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$$

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\underbrace{\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}})$$

Trainable Network, Shared Across All Timesteps



Learning the Denoising Model

- Denoising models are trained with variational upper bound (negative ELBO), as VAEs

$$\mathbb{E}_{q(\mathbf{x}_0)} [-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] =: L$$

- which derives to:

$$L = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \right]_{L_0}$$

- tractable posterior distribution (closed-form)

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

$$\text{where } \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{1 - \beta_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t \text{ and } \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

Learning the Denoising Model

- Denoising models are trained with variational upper bound (negative ELBO), as VAEs

$$\mathbb{E}_{q(\mathbf{x}_0)} [-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] =: L$$

- which derives to:

$$L = \underbrace{\mathbb{E}_q [D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))]}_{L_T \text{ constant}} + \underbrace{\sum_{t>1} D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_{t-1}} \underbrace{]}_{L_0 \text{ Scaling}}$$

- tractable posterior distribution (closed-form)

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

$$\text{where } \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{1 - \beta_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t \text{ and } \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

Learning the Denoising Model

- Denoising models are trained with variational upper bound (negative ELBO), as VAEs

$$\mathbb{E}_{q(\mathbf{x}_0)} [-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] =: L$$

- which derives to:

$$L = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

- tractable posterior distribution (closed-form)

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

$$\text{where } \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{1 - \beta_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t \text{ and } \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

Parameterizing the Denoising Model

- KL divergence has a simple form between Gaussians

$$L_{t-1} = D_{\text{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)) = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right] + C$$

- Recall that: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
- Trainable network predicts the noise mean

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right)$$

- Final Objective

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\frac{\beta_t^2}{2\sigma_t^2 (1 - \beta_t) (1 - \bar{\alpha}_t)} \|\epsilon - \underbrace{\epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)}_{\mathbf{x}_t}\|^2 \right] + C$$

Simplified Training Objective

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\underbrace{\frac{\beta_t^2}{2\sigma_t^2 (1 - \beta_t) (1 - \bar{\alpha}_t)}}_{\lambda_t} \left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

- λ_t ensures the weighting for correct maximum likelihood estimation
- In DDPM, this is further simplified to:

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1, T)} \left[\left\| \epsilon - \epsilon_\theta \left(\underbrace{\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t}_{\mathbf{x}_t} \right) \right\|^2 \right]$$

Summary: Training and Sampling

Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on $\nabla_{\theta} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t)\|^2$
 - 6: **until** converged
-

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 2: **for** $t = T, \dots, 1$ **do**
 - 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
 - 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
 - 5: **end for**
 - 6: **return** $\mathbf{x}_0$
-

Summary: Noise Schedule

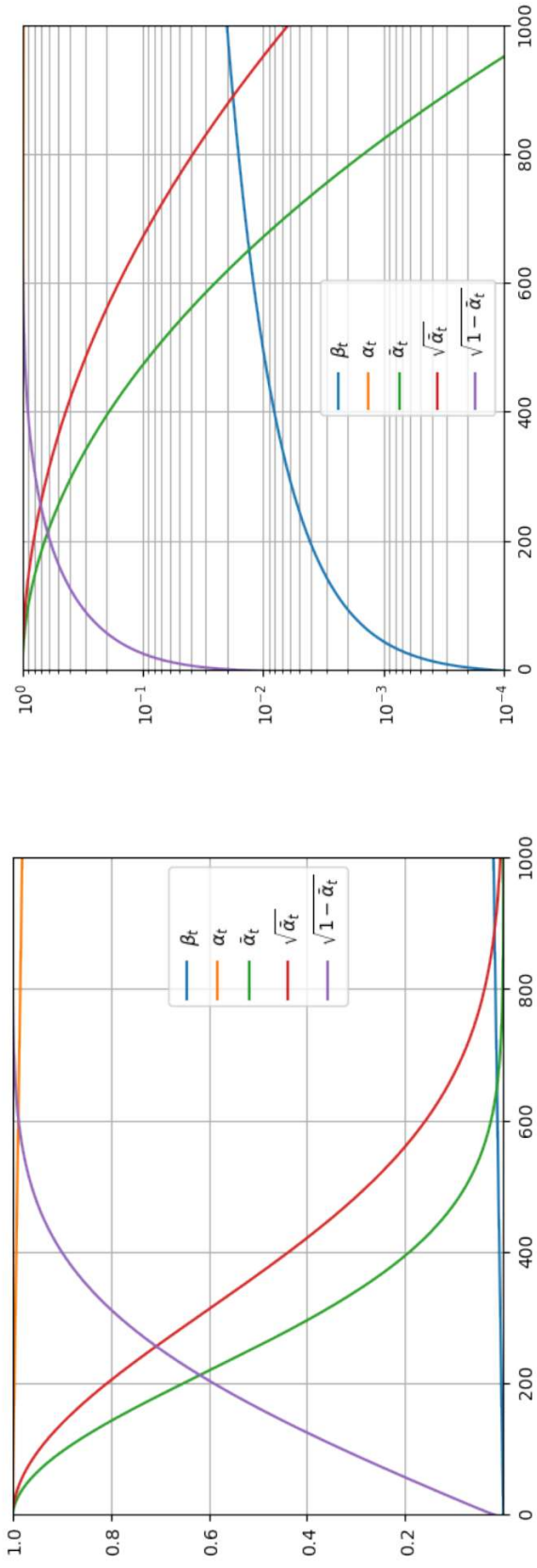


Figure 2: Parameter values for $\beta = [10^{-4}, 0.02]$ over 1000 time steps t using a linear schedule. The information in the two figures are the same, but the right-hand side uses log-scale on the y -axis to show the speed of which $\bar{\alpha}_t$ goes towards zero.

Connection with Hierarchical VAEs

- Diffusion models are special case of Hierarchical VAEs
 - Fixed inference models in forward process
 - Latent variables have same dimension as data
 - ELBO is decomposed to each timestep: faster to train
 - Model is trained with some weighting of ELBO

